

Snake Game Project Report

Development Report

November 15, 2025

1 Project Introduction

This project implements a classic Snake game using Python and Pygame. The Snake game is an interactive gameplay experience where the player controls a snake that moves on a 2D grid. The objective is to eat food items to grow the snake while avoiding collisions with walls and the snake's own body.

1.1 Key Features

- Interactive snake gameplay with arrow key controls
- Real-time score tracking during gameplay
- Collision detection for walls and self-collision
- Difficulty level configuration (Easy, Medium, Hard)
- Game over detection with restart option (press C to restart or Q to quit)

1.2 Technology Stack

- Language: Python 3.x
- Graphics Library: Pygame
- Game Loop: Custom event-driven architecture

2 AI Prompts Used During Development

The following AI prompts were used to develop and improve the Snake Game:

2.1 Prompt 1: Initial Game Structure

“Create a Python Snake game using Pygame with basic gameplay mechanics including snake movement, food eating, and collision detection.”

This prompt guided the creation of the initial game structure with core mechanics.

2.2 Prompt 2: Score Display Enhancement

“Add a score display function to the Snake game that shows the current score in real-time during gameplay.”

This prompt led to the implementation of the score tracking system.

2.3 Prompt 3: Code Refactoring

“Extract the collision detection logic into a separate reusable function to improve code maintainability.”

This prompt resulted in better code organization through the creation of a dedicated collision detection function.

2.4 Prompt 4: Difficulty Levels

“Add difficulty level configuration to the Snake game with Easy, Medium, and Hard modes.”

This prompt introduced game configuration capabilities and difficulty parameters.

2.5 Prompt 5: Documentation

“Write comprehensive README documentation for the Snake Game project including features, installation, controls, and game rules.”

This prompt produced the complete project documentation.

3 Development History

The project was developed through the following commits (last 5 commits from git log):

```
473d7f0 docs: add comprehensive project documentation
69f2656 feat: add difficulty level configuration
d852ded refactor: extract collision detection into separate function
ced2ede feat: add score display during gameplay
7e577e7 Add initial Snake game code
```

Each commit represents a meaningful development step:

1. Initial Snake game code with basic gameplay mechanics
2. Added score display functionality during gameplay
3. Refactored collision detection into a separate function
4. Introduced difficulty level configuration system
5. Added comprehensive project documentation

4 Game Architecture

4.1 Core Components

- **Game Loop:** Main game loop that handles events, updates game state, and renders graphics
- **Snake Management:** List-based representation of the snake body segments
- **Collision Detection:** Functions to check wall collisions and self-collisions
- **Rendering System:** Pygame-based graphics rendering for snake, food, and UI
- **Score Tracking:** Real-time score tracking and display system

4.2 Key Functions

- `message()`: Displays text messages on screen
- `display_score()`: Shows current score during gameplay
- `check_game_over()`: Checks for collision conditions
- `gameLoop()`: Main game execution loop
- `get_difficulty()`: Returns difficulty level settings

5 Game Controls

Key	Action
Arrow Up	Move snake upward
Arrow Down	Move snake downward
Arrow Left	Move snake left
Arrow Right	Move snake right
Q	Quit after game over
C	Play again after game over

6 Conclusion

The Snake Game project demonstrates effective use of Python and Pygame to create an interactive game experience. Through systematic development with meaningful commits and AI assistance, the project evolved from basic gameplay mechanics to a feature-complete game with difficulty levels, score tracking, and comprehensive documentation.

The modular design and clear separation of concerns make the codebase maintainable and extensible for future enhancements.